OVERVIEW
YOLO & TRANSFORMER

MAIN4 - Han, Mohamed Saad, Adrien, Victor

YOLO

Classifiers

- Image = 3D tensor (W*H*C)
- Pixel = RGB values
- CNN = kernels (KR, KG, KB)

→ values of importance

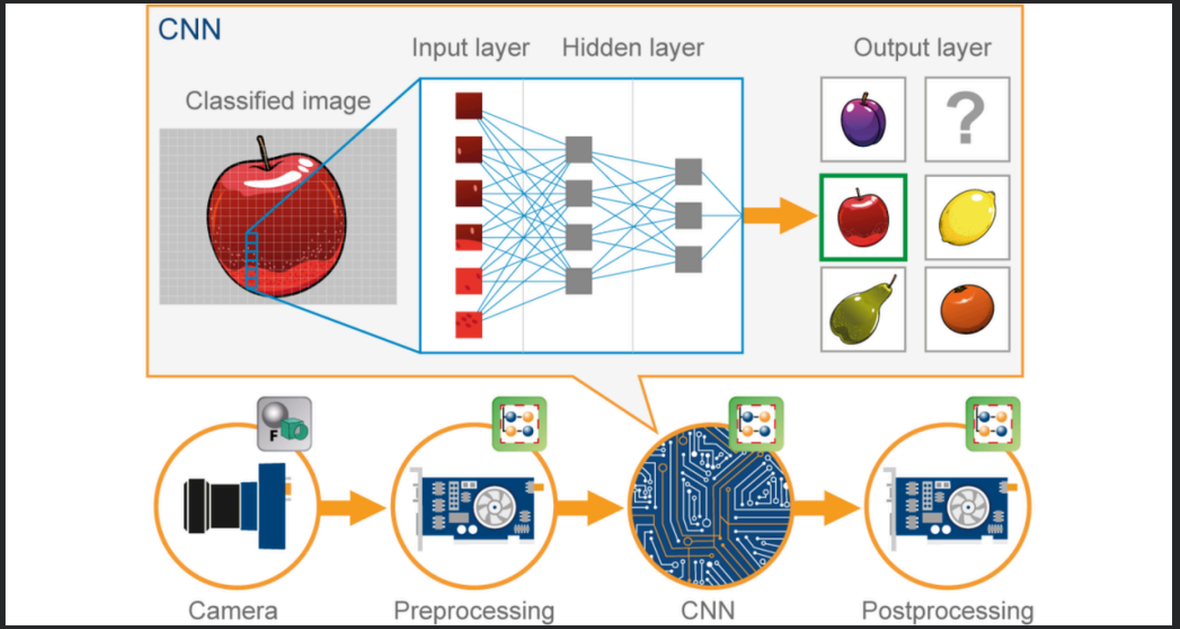
- Output formula

→ feature map

- Activation function (ReLU, Sigmoid)

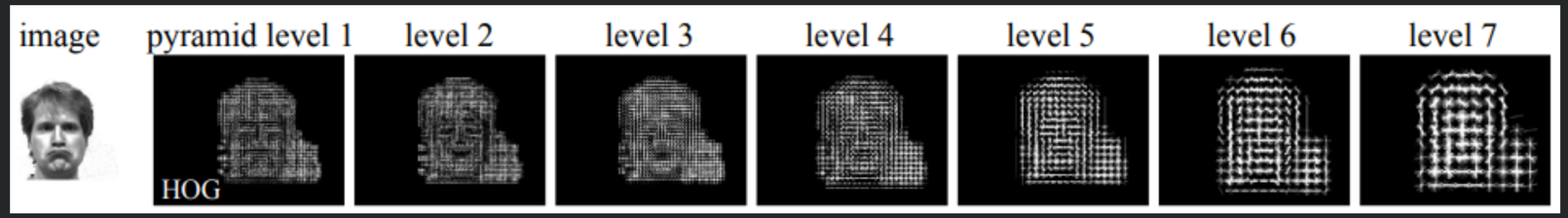
→ nonlinearity

=> Predicts a class label



Layer	What the pixel values represent
Input layer	Physical light intensities (0–255 per channel)
Conv Layer 1	Weighted combinations of R/G/B values : edge maps, texture maps
Conv Layer 3–5	Abstract features : curves, corners, blobs
Deep layers	Semantic parts : “dog ear”, “wheel”, “face”
Final layer	Class logits : “dog”, “car”

YOLO

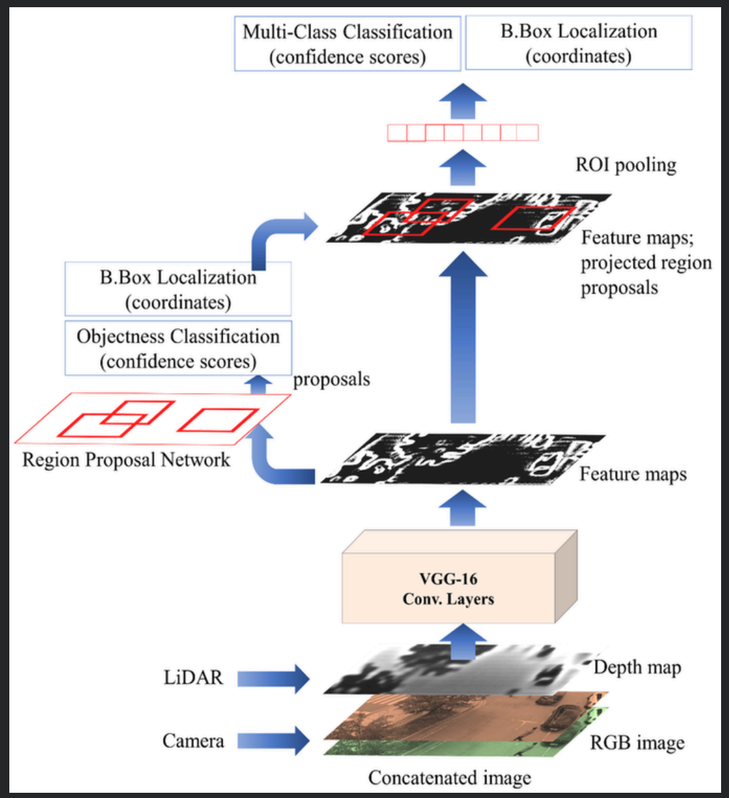


https://arxiv.org/pdf/1409.5403

Image detection methods

- DPM (Deformable Parts Models)
  - Sliding window
  - Many classifiers on each window

- R-CNN (Region-based CNN)
  - Region proposal (selective search) → bounding boxes
  - Classifier
  - Post-processing



https://www.researchgate.net/publication/349712442_RGDiNet_Efficient_Onboard_Object_Detection_with_Faster_R-CNN_for_Air-to-Ground_Surveillance

YOLO

Principle

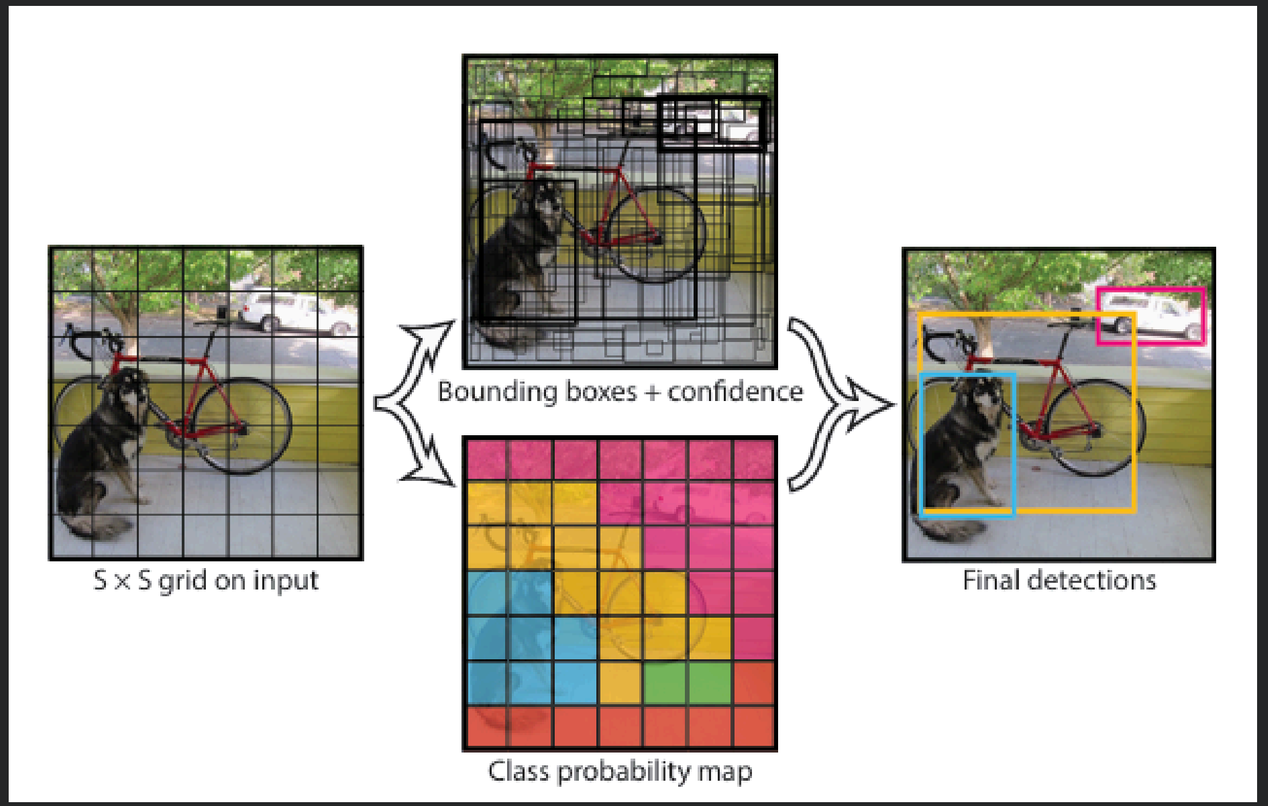
- Single regression problem

→ all in once

- Image = S*S grid
- B boxes with a score confidence

→ B=2 for Yolo1

- 5 predictions per box → x,y,w,h,confidence
- C conditionnal class probabilities

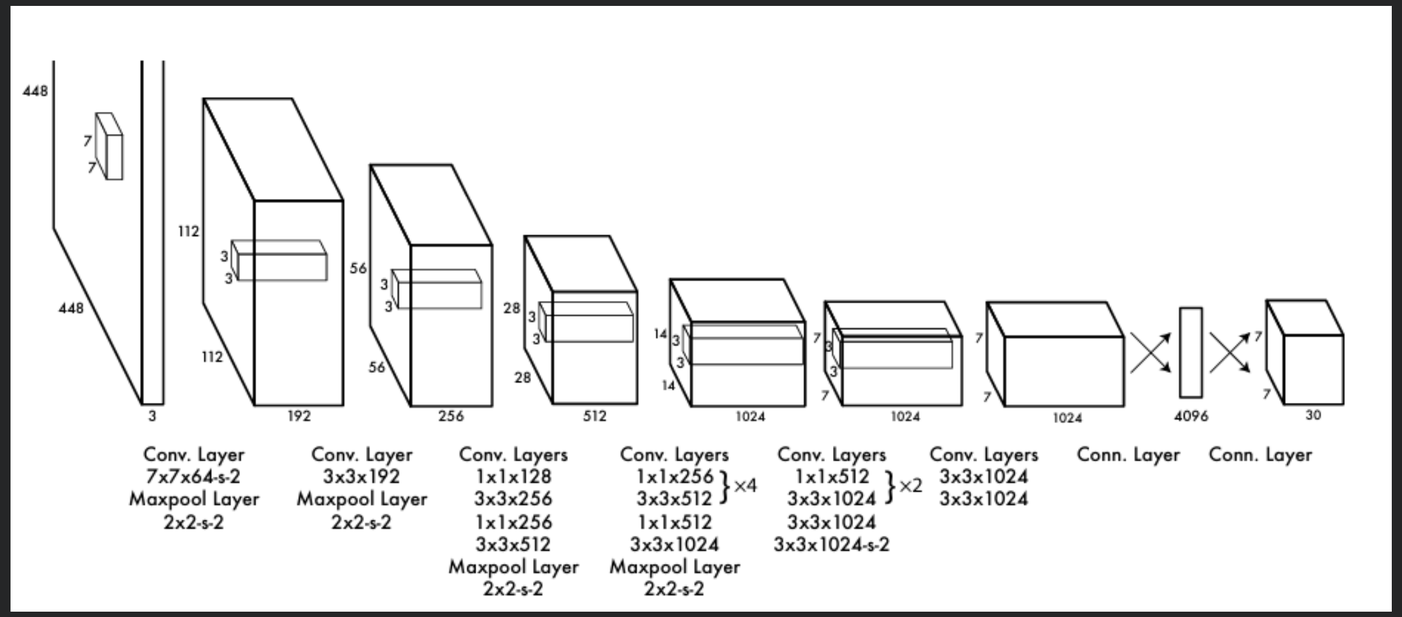


https://www.researchgate.net/publication/349712442_RGDiNet_Efficient_Onboard_Object_Detection_with_Faster_R-CNN_for_Air-to-Ground_Surveillance

YOLO

CNN - Architecture

- 24 convolutional layers
- 2 fully connected layers
- Final output = 7*7*30 tensor of predictions



https://www.researchgate.net/publication/349712442_RGDiNet_Efficient_Onboard_Object_Detection_with_Faster_R-CNN_for_Air-to-Ground_Surveillance

YOLO

Performance (Pictures)

Real-time detectors

→ faster & more accurate in time

- YOLO1: 45 FPS, 63.4 mAP
- FAST YOLO: 155 FPS, 52.7 mAP
- DPM (100Hz): 100 FPS, 16.0 mAP
- R-CNN: 6 FPS, 53.5 mAP

→ Hard to improve Less Than real-time detectors in time-accuracy

Real-Time Detectors	Train	mAP	FPS
100Hz DPM [31]	2007	16.0	100
30Hz DPM [31]	2007	26.1	30
Fast YOLO	2007+2012	52.7	**155**
YOLO	2007+2012	**63.4**	45
Less Than Real-Time			
Fastest DPM [38]	2007	30.4	15
R-CNN Minus R [20]	2007	53.5	6
Fast R-CNN [14]	2007+2012	70.0	0.5
Faster R-CNN VGG-16[28]	2007+2012	73.2	7
Faster R-CNN ZF [28]	2007+2012	62.1	18
YOLO VGG-16	2007+2012	66.4	21

https://www.researchgate.net/publication/349712442_RGDiNet_Efficient_Onboard_Object_Detection_with_Faster_R-CNN_for_Air-to-Ground_Surveillance

YOLO

Performance (Artwork)

Yolo generalizes better than other detection systems

- YOLO1: 59.2 AP, 45 AP
- R-CNN : 54.2 AP, 26 AP
- DPM : 37.8 AP, 32 AP

Precision = TP / (TP + FP)

Recall = TP / (TP + FN)

F1 = 2 * Precision * Recall / (Precision + Recall)

	VOC 2007	Picasso		People-Art
	AP	AP	Best F_1	AP
YOLO	**59.2**	**53.3**	**0.590**	**45**
R-CNN	54.2	10.4	0.226	26
DPM	43.2	37.8	0.458	32
Poselets [2]	36.5	17.8	0.271	
D&T [4]	-	1.9	0.051	

https://www.researchgate.net/publication/349712442_RGDiNet_Efficient_Onboard_Object_Detection_with_Faster_R-CNN_for_Air-to-Ground_Surveillance

TRANSFORMERS

- Avant 2017 : domination de RNN, LSTM, GRU pour le NLP
- Ces modèles traitent les tokens(inputs) séquentiellement → limite la parallélisation
- Attention existait déjà, mais toujours en complément d'un RNN
- Objectif : abolir totalement la récursivité

TRANSFORMERS

Architecture d’un Transformer

- Architecture Encoder-Decoder empilée (N=6 couches)
- Chaque bloc = Attention + Feed Forward + Residual + Norm
- Encodeur = Self-Attention uniquement
- Décodeur = Self-Attention + Encoder-Decoder Attention

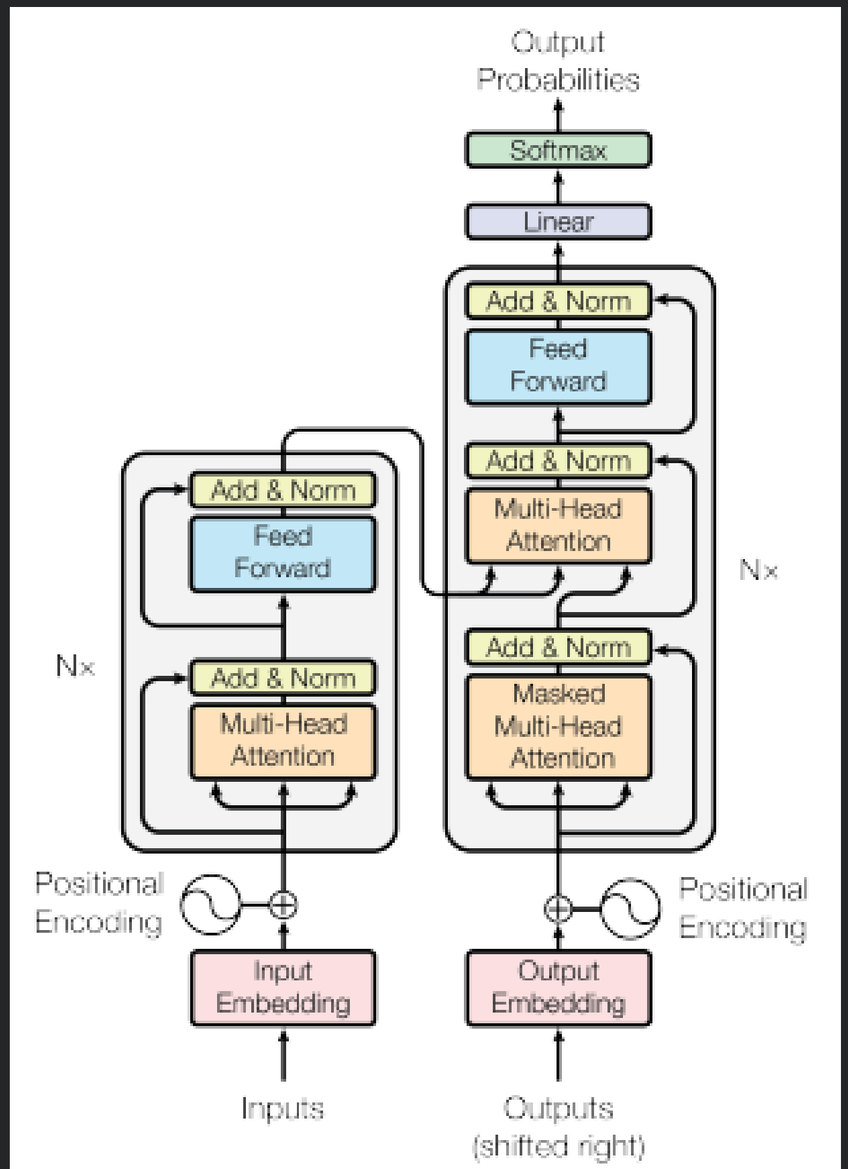


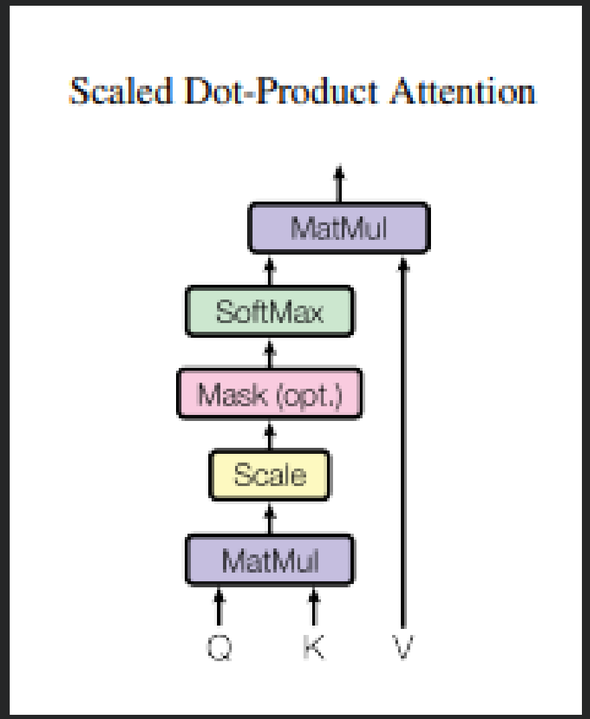
Figure 1: The Transformer - model architecture.

https://arxiv.org/pdf/1706.03762

TRANSFORMERS

Qu’est ce que Self-Attention ?

- Chaque position d’une séquence peut regarder toutes les autres positions
- Représente les dépendances entre chaque input
- Q = Query / K = Key / V = Value
- On divise par √dk → stabilise gradients
- Softmax crée distribution d’attention



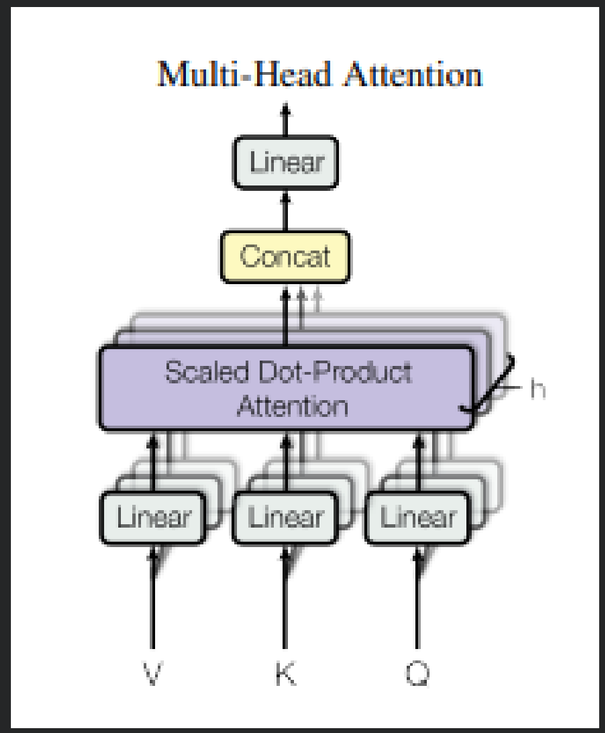
$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

https://arxiv.org/pdf/1706.03762

TRANSFORMERS

Multi-Head Attention

- On répète l'attention h = 8 fois en parallèle
- Chaque tête apprend une relation différente
- Combine en concaténation puis projection finale



$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O$$
$$\text{where head}_\text{i} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

https://arxiv.org/pdf/1706.03762

TRANSFORMERS

Comparaison

- RNN : séquentiel O(n)
- CNN : plusieurs couches pour connecter loin
- Self-Attention : toutes positions connectées en 1 opération
- Complexité par couche = O(n²·d) mais séquentiel = O(1)
- Meilleur pour apprendre dépendances longues
- n : longueur de la séquence

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

https://arxiv.org/pdf/1706.03762

TRANSFORMERS

Résultats après entraînement

- EN→DE : 28.4 BLEU
- EN→FR : 41.8 BLEU
- Entraînement moins coûteux que ConvS2S/ GNMT
- Transformer 12H
- Transformers (big) : 3.5 jours
- 8 P100 Nvidia GPU

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	**$3.3 \cdot 10^{18}$**	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

https://arxiv.org/pdf/1706.03762

TRANSFORMERS

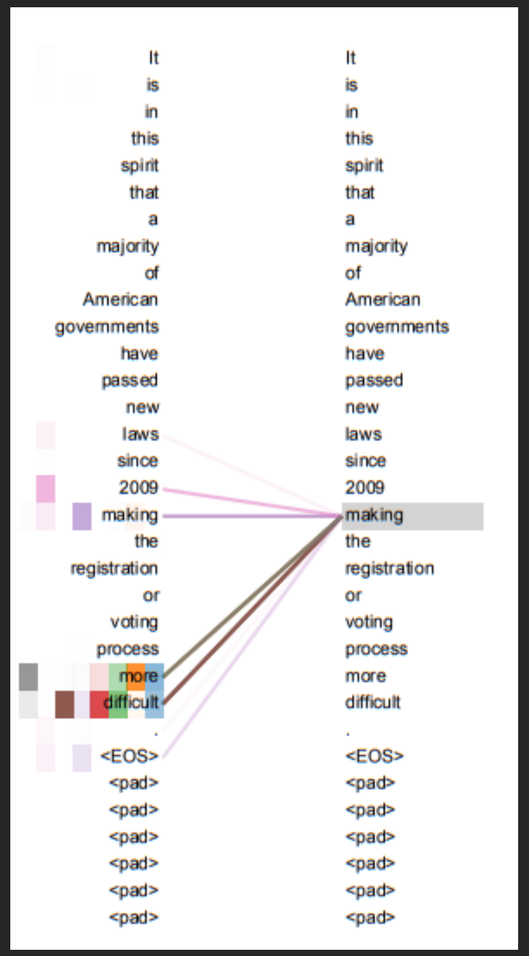
Les modèles

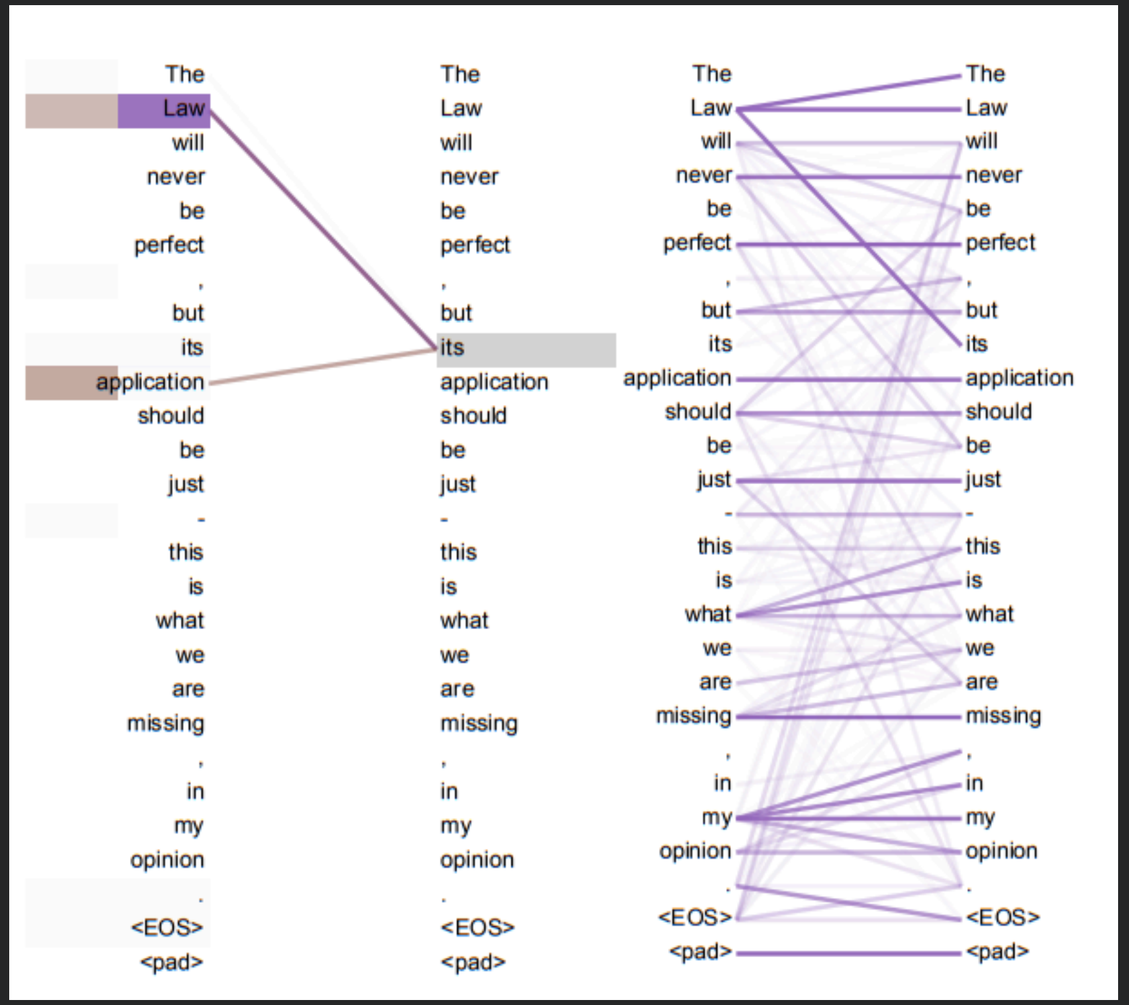
	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)	positional embedding instead of sinusoids									4.92	25.7	
big	6	1024	4096	16			0.3		300K	**4.33**	**26.4**	213

https://arxiv.org/pdf/1706.03762

TRANSFORMERS

Visualisation





https://arxiv.org/pdf/1706.03762